mp3
Audio Restoration

Hayden Hawley

Overview

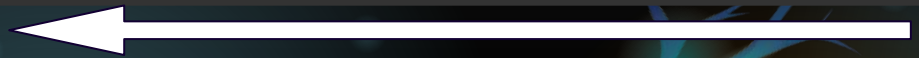
- I built a model that restores audio quality in low-bitrate MP3 files.
- The easiest way to explain what the model does is to first explain what it doesn't do: this model is not **upsampling** audio. Upsampling involves taking a low sample rate audio file and filling in information where there was none. That would be the equivalent of AI upscaling an image by adding new pixels, increasing the resolution (dimensions) of the image.
- This model, on the other hand, is an MP3 **decompressor**. Rather than increasing the sample rate, the goal is to increase the *bitrate*, similar to cleaning or "de-noising" a heavily compressed JPEG instead of just enlarging it. When MP3s are compressed at low bitrates (like 64 or 96 kbps), the audio sounds muffled and tinny due to compression artifacts. These are especially noticeable in older music libraries, internet rips, or archival recordings. The goal here is to enhance such degraded MP3s so they sound closer to the original high-quality versions.

I’m doing the audio equivalent of this:

Quality 100 | Jpeg compression

Quality 50 | Jpeg compression

Quality 10 | Jpeg compression

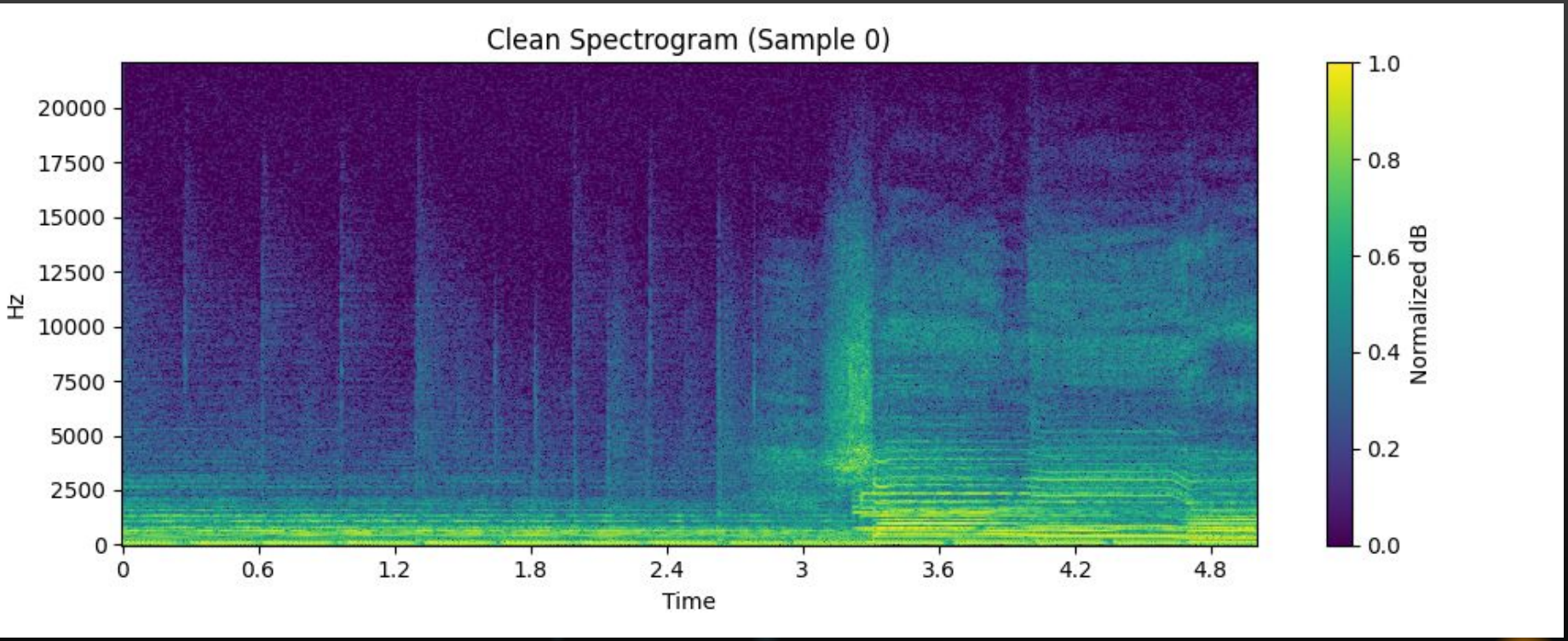


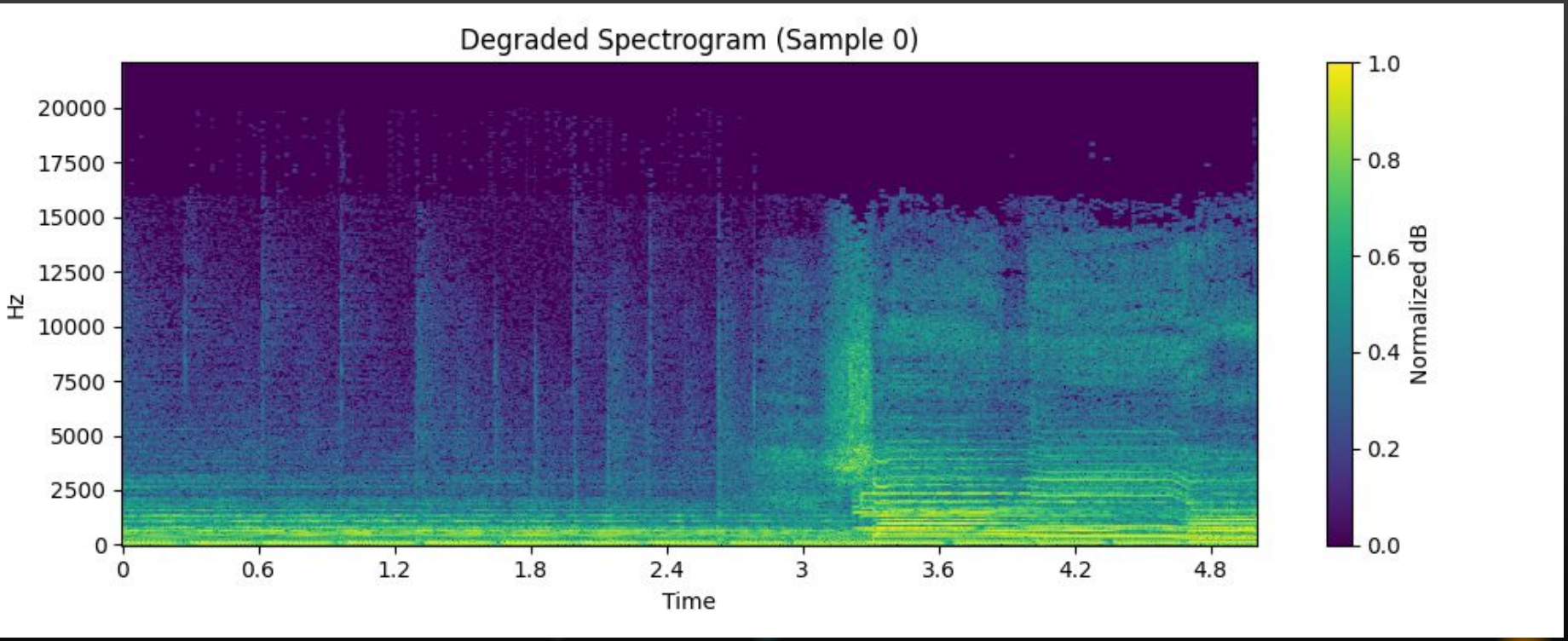
Overview

- While plenty of research has been done to improving **sample rate** (for things like speech clarity) this approach focuses on restoring **bitrate** in music which is a less explored area. The utility of this model is to clean up music such that it’s audibly clearer and more enjoyable to listen to; whether the model achieves this is subjective, but supported by measuring how close the "restored" samples are to their original clean counterparts.

Dataset & Preprocessing

- I used a supervised learning setup with pairs of audio clips: one “clean” high-quality version and one “degraded” low-quality version (created by compressing and then decoding the original at a low bitrate: 32kbps). This gives the model clear before-and-after examples, so it learns not just to memorize the waveform, but to recognize patterns tied to compression artifacts in order to reverse them.
- Rather than feeding in raw waveforms, the model uses spectrograms, which are 2D representations of how sound energy is distributed across different frequencies over time—like a heat map for audio. This allows the model to spot where frequencies are missing or distorted. Convolutional neural networks then denoise and restore lost detail.


Clean Spectrogram (Sample 0)
Hz
20000
17500
15000
12500
10000
7500
5000
2500
0
0
0.6
1.2
1.8
2.4
3
3.6
4.2
4.8
Time
1.0
0.8
0.6
0.4
0.2
0.0
Normalized dB


Degraded Spectrogram (Sample 0)
Hz
20000
17500
15000
12500
10000
7500
5000
2500
0
0
0.6
1.2
1.8
2.4
3
3.6
4.2
4.8
Time
Normalized dB
1.0
0.8
0.6
0.4
0.2
0.0

Dataset & Preprocessing

- The dataset consists of paired .wav audio files: one degraded (saved as a 32kbps .mp3 and converted back to .wav) and one clean (the original, used as the label). Each file was converted into a magnitude spectrogram using Short-Time Fourier Transform (STFT). The model takes these spectrograms as input, while the original clean spectrograms serve as the target. During inference, phase information is preserved and used to reconstruct the waveform via inverse STFT (iSTFT).

Final Model

- U-Net (a CNN with encoder and decoder shapes)
  - Encoder: compress the input to capture key features
  - Decoder: reconstruct a cleaned-up version using those features
  - Skip connections: help the decoder recover lost detail by copying encoder outputs into the decoder
- The input is a spectrogram chunk: a 513×[time]×1 grayscale image:
  - 513 = frequency bins (from STFT)
  - time = variable, 44.1kHz (samples per second)
  - 1 = single channel (like a grayscale image)

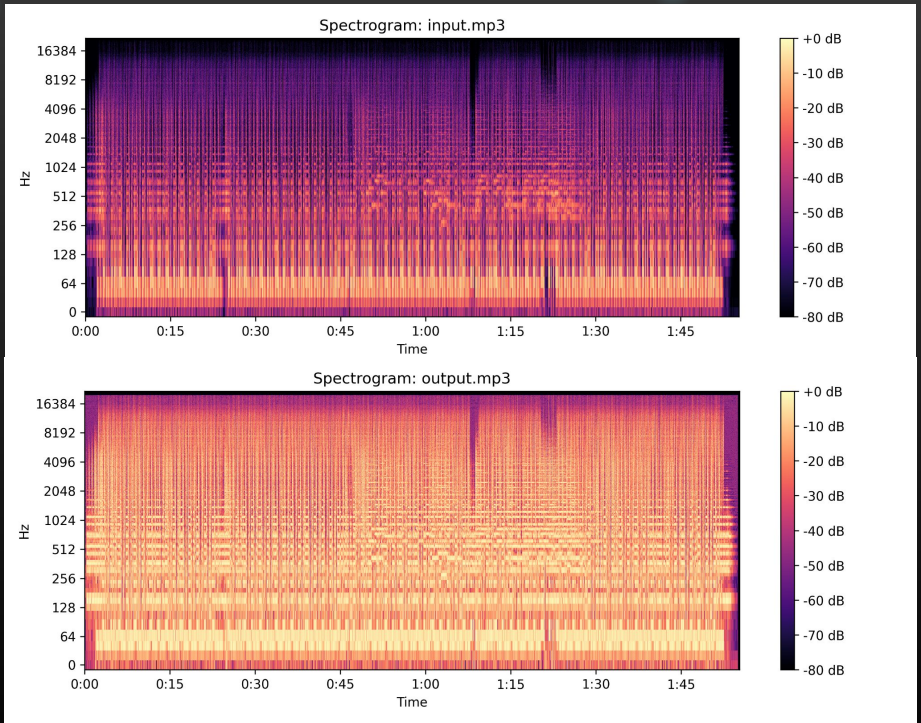
Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 513, None, 1)	0	-
conv2d (Conv2D)	(None, 513, None, 64)	640	input_layer[0][0]
batch_normalization (BatchNormalization)	(None, 513, None, 64)	256	conv2d[0][0]
leaky_re_lu (LeakyReLU)	(None, 513, None, 64)	0	batch_normalization[0][…
average_pooling2d (AveragePooling2D)	(None, 257, None, 64)	0	leaky_re_lu[0][0]
conv2d_1 (Conv2D)	(None, 257, None, 128)	73,856	average_pooling2d[0][0]
batch_normalization_1 (BatchNormalization)	(None, 257, None, 128)	512	conv2d_1[0][0]
leaky_re_lu_1 (LeakyReLU)	(None, 257, None, 128)	0	batch_normalization_1[0…
average_pooling2d_1 (AveragePooling2D)	(None, 129, None, 128)	0	leaky_re_lu_1[0][0]
conv2d_2 (Conv2D)	(None, 129, None, 256)	295,168	average_pooling2d_1[0][…
batch_normalization_2 (BatchNormalization)	(None, 129, None, 256)	1,024	conv2d_2[0][0]
leaky_re_lu_2 (LeakyReLU)	(None, 129, None, 256)	0	batch_normalization_2[0…
conv2d_3 (Conv2D)	(None, 129, None, 256)	590,080	leaky_re_lu_2[0][0]
batch_normalization_3 (BatchNormalization)	(None, 129, None, 256)	1,024	conv2d_3[0][0]
leaky_re_lu_3 (LeakyReLU)	(None, 129, None, 256)	0	batch_normalization_3[0…
up_sampling2d (UpSampling2D)	(None, 258, None, 256)	0	leaky_re_lu_3[0][0]
cropping2d (Cropping2D)	(None, 257, None, 256)	0	up_sampling2d[0][0]
concatenate (Concatenate)	(None, 257, None, 384)	0	cropping2d[0][0], leaky_re_lu_1[0][0]
conv2d_4 (Conv2D)	(None, 257, None, 128)	442,496	concatenate[0][0]
batch_normalization_4 (BatchNormalization)	(None, 257, None, 128)	512	conv2d_4[0][0]
leaky_re_lu_4 (LeakyReLU)	(None, 257, None, 128)	0	batch_normalization_4[0…
up_sampling2d_1 (UpSampling2D)	(None, 514, None, 128)	0	leaky_re_lu_4[0][0]
cropping2d_1 (Cropping2D)	(None, 513, None, 128)	0	up_sampling2d_1[0][0]
concatenate_1 (Concatenate)	(None, 513, None, 192)	0	cropping2d_1[0][0], leaky_re_lu[0][0]
conv2d_5 (Conv2D)	(None, 513, None, 64)	110,656	concatenate_1[0][0]
batch_normalization_5 (BatchNormalization)	(None, 513, None, 64)	256	conv2d_5[0][0]
leaky_re_lu_5 (LeakyReLU)	(None, 513, None, 64)	0	batch_normalization_5[0…
conv2d_6 (Conv2D)	(None, 513, None, 1)	65	leaky_re_lu_5[0][0]

Total params: 1,516,545 (5.79 MB)

Training Progress

- Training was done in small chunks to manage memory and track progress incrementally. Each chunk processes a few samples with early stopping based on validation loss. The model uses Mean Absolute Error (MAE) and Mean Squared Error (MSE) as training metrics.

Growing pains



This was using a no-op baseline model so these two should be identical; my mp3 decompression script was broken, not the model

Results

Model A:

- Trainable params: 738,561 (2.82 MB)
- loss: 0.0035 - mae: 0.0431 - val_loss: 0.0302 - val_mae: 0.1220

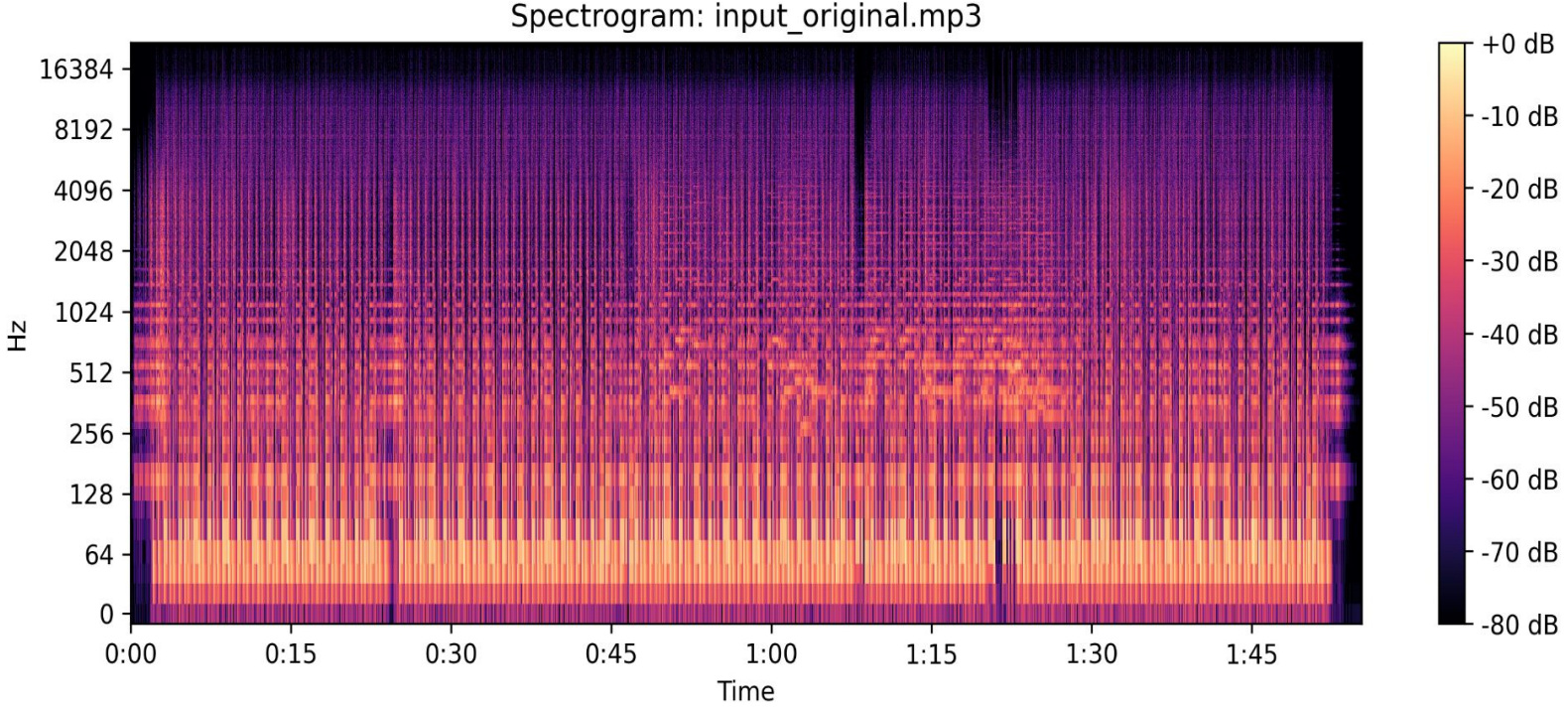
Model B:

- Trainable params: 924,161 (3.53 MB)
- loss: 0.0021 - mae: 0.0309 - val_loss: 0.0174 - val_mae: 0.1027

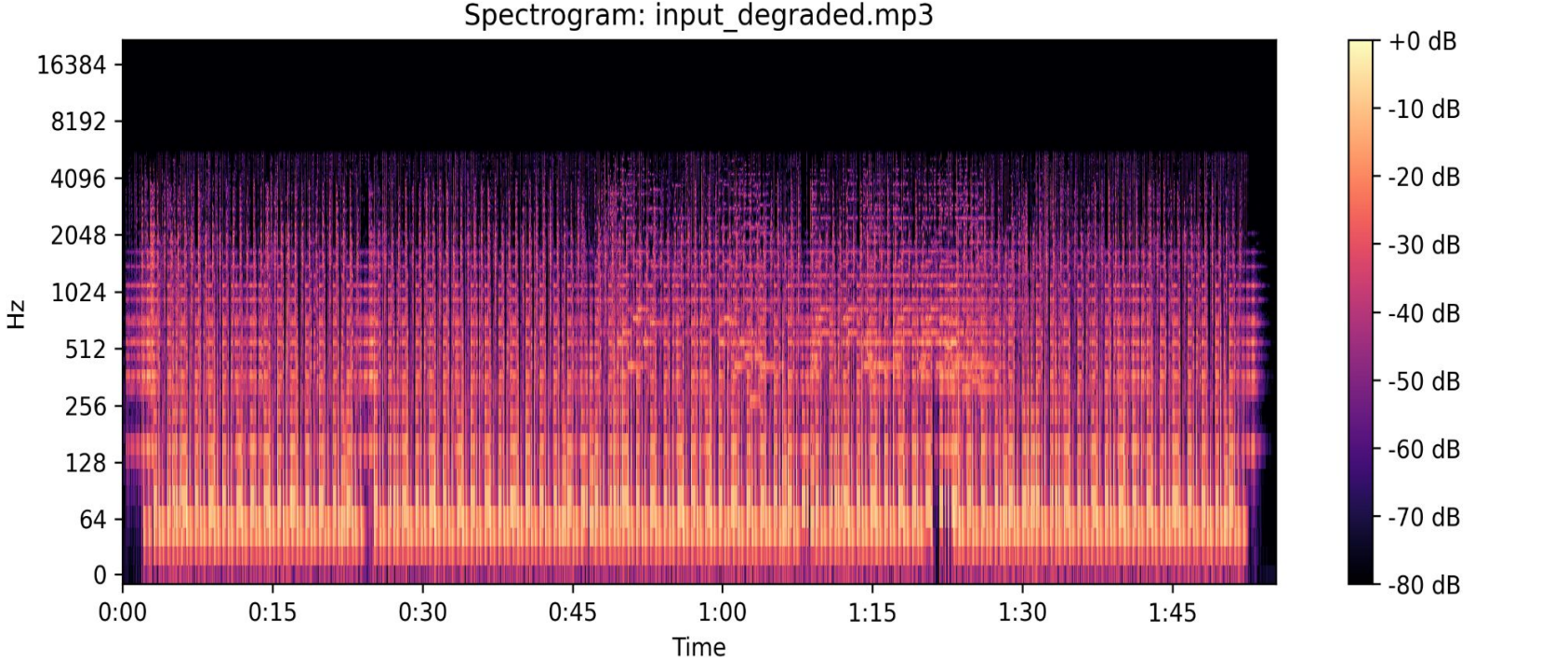
Model C:

- Trainable params: 1,514,753 (5.78 MB)
- loss: 0.0015 - mae: 0.0318 - val_loss: 0.0115 - val_mae: 0.1021

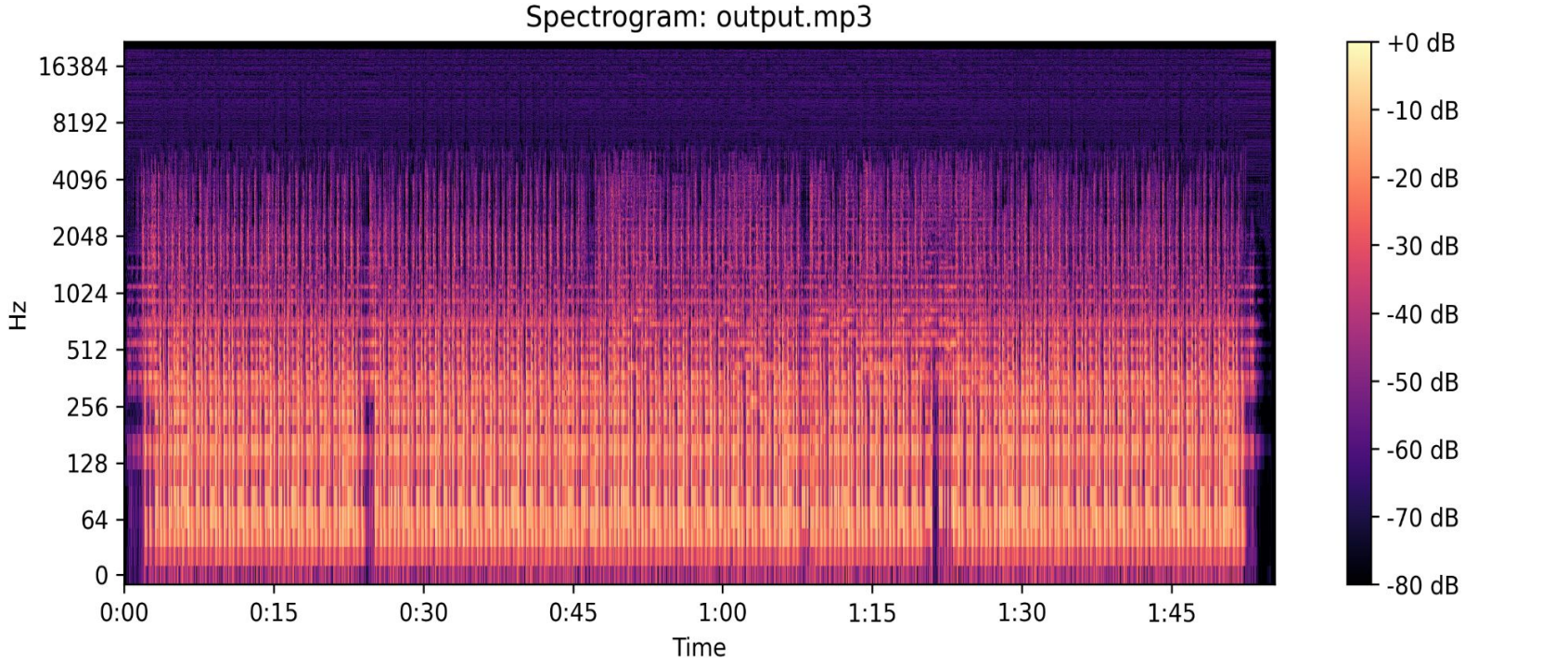
Results


Spectrogram: input_original.mp3
16384
8192
4096
2048
1024
512
256
128
64
0
Hz
0:00
0:15
0:30
0:45
1:00
1:15
1:30
1:45
Time
+0 dB
-10 dB
-20 dB
-30 dB
-40 dB
-50 dB
-60 dB
-70 dB
-80 dB

Results


Spectrogram: input_degraded.mp3
16384
8192
4096
2048
1024
512
256
128
64
0
Hz
0:00
0:15
0:30
0:45
1:00
1:15
1:30
1:45
Time
+0 dB
-10 dB
-20 dB
-30 dB
-40 dB
-50 dB
-60 dB
-70 dB
-80 dB

Results


Spectrogram: output.mp3
16384
8192
4096
2048
1024
512
256
128
64
0
Hz
0:00
0:15
0:30
0:45
1:00
1:15
1:30
1:45
Time
+0 dB
-10 dB
-20 dB
-30 dB
-40 dB
-50 dB
-60 dB
-70 dB
-80 dB

https://youtu.be/9WgnsgI6f2s?si=NpaKcVZFkEAFeF9X

Why I think it doesn’t work well (yet)

- Model tries to fill in missing high-frequency data, and does so with random noise. Doing so gives it a “better” score compared to the compressed audio, but it’s not very close and actually *sounds* even worse. 😫

What I’ll try in the future:

- Train on log-magnitude or spectral convergence instead of linear MSE (should be closer to human perception)
- Don’t reuse the degraded phase verbatim; random magnitudes + wrong phase = noise. Run a few iterations of Griffin–Lim on reconstructed complex spectrogram to get a smoother, more coherent waveform
- Under-penalize lower bins (low frequency areas where mp3 compression doesn't really affect)